Team Name : Uttirn Gyan , Jibran , Naren Jindal

Problem : MNIST classification using QML and comparing it with Classical ML

Final Remarks on the Problem

This report talks contrasts Classical ML and QML for classifying the MNIST Dataset by using various methods such as SVM , Classical CNN (for Classical methods) and Regular QNN using Rx , Ry and entanglement gates as well as using Quanvolution (for Quantum Model)

USING CLASSICAL ML AND DL MODELS

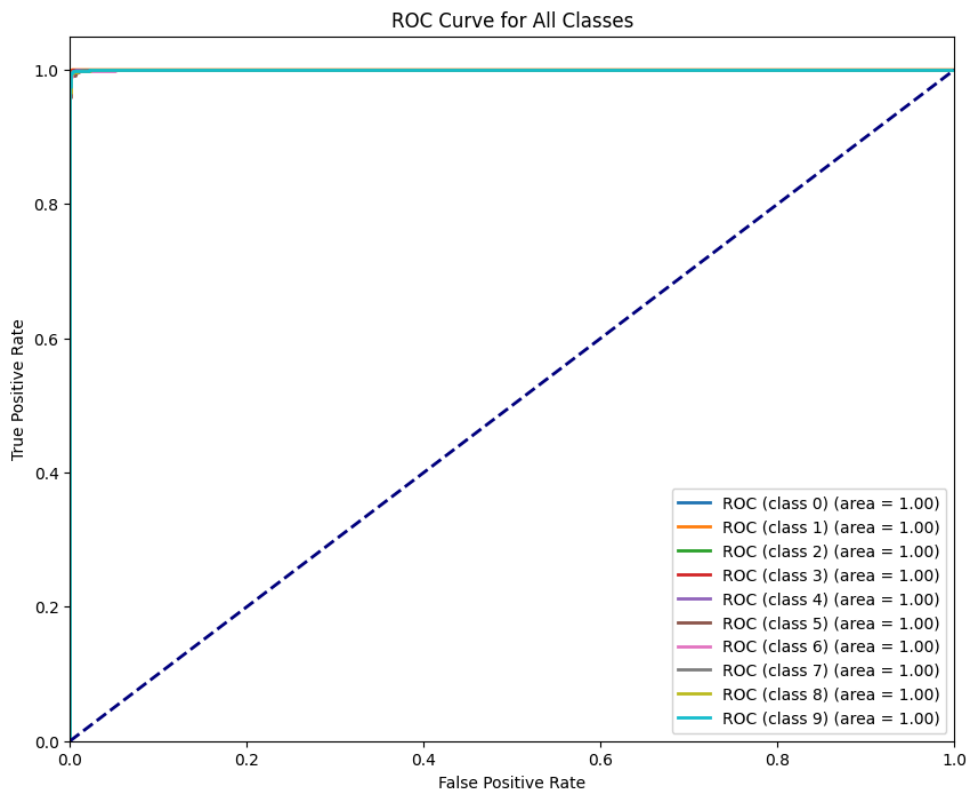
USING CNN

The CNN model architecture model :

```
 nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2),
nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2)
nn.Linear(64 * 7 * 7, 128),
nn.ReLU(),
nn.Linear(128, 10)
```

Performance on the basis of accuracy and ROC curve :

Accuracy (10000 test images): 99.18%



This clearly shows that the CNN model performed very well ,

Note : This was the most basic CNN model yet it performed very well

USING SVM

The SVM is used along with feature extractor CNN :

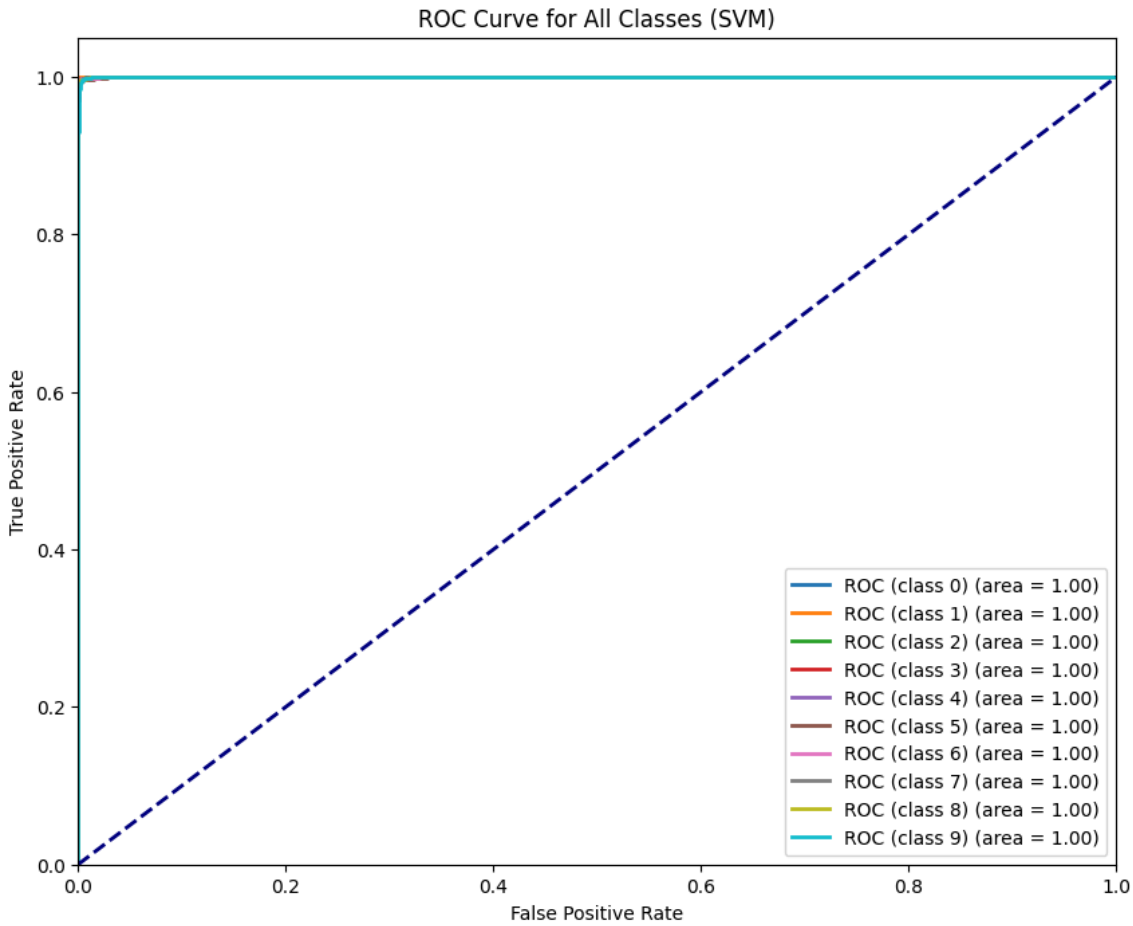
```
Conv2d(1, 32, kernel_size=3, stride=1, padding=1),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2),
nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2)
```

The configuration the the SVM model are as follows :

```
SVC(kernel='linear', random_state=42, probability=True)
```

The ROC and Accuracy are as follows :

Accuracy (SVM on CNN features): 0.9924



Even the SVM model performed very well , this shows that classical computers are the best for datasets such as MNIST

USING QUANTUM COMPUTING (QML MODELS)

All the models were run on ‘default.qubit’ from pennylane

BASIC QML MODEL :

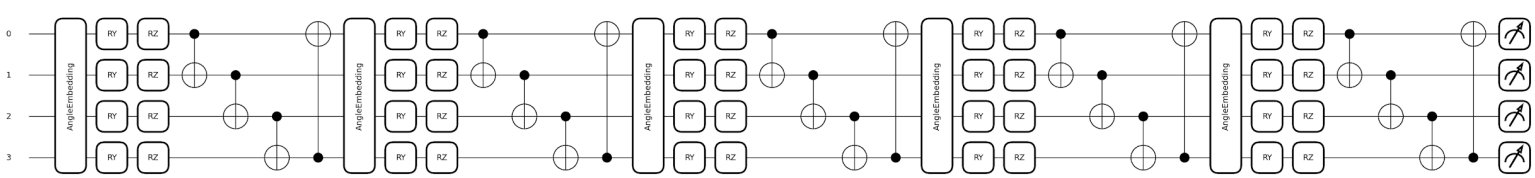
The architecture is as follows :

```
x = x.view(-1, 28 * 28)
x = torch.relu(self.fc1(x))
x = self.qnn(x)
x = torch.relu(x)
x = torch.relu(self.fc2(x))
x = self.fc3(x)
```

Wherein :

```
self.fc1 = nn.Linear(28 * 28, n_qubits)
        self.qnn = quantum_layer
        self.fc2 = nn.Linear(n_qubits * 2, 32)  # Note: quantum output size doubled (Z and X)
        self.fc3 = nn.Linear(32, 10)
```

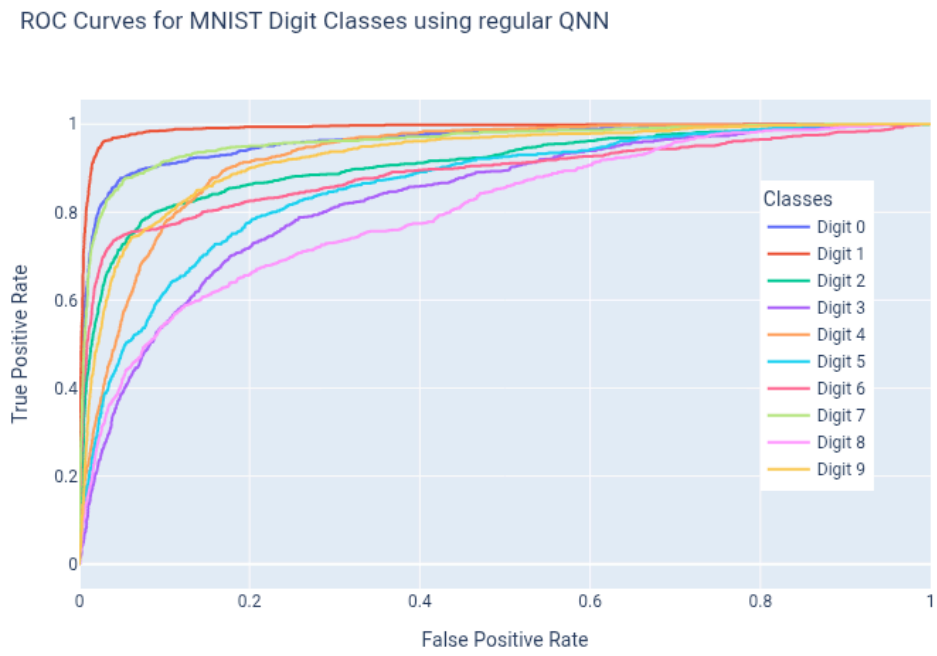
The Quantum Layer looked like this :



The Accuracy and ROC are as follows :

Test Accuracy: 64.07%

Note : If we look at the losses of the last 5 epochs [0.202 , 0.2 , 0.228 , 0.208 , 0.18] this shows that the model clearly Over fit owning to less examples being trained for due to lack of computational power



The model was easily able to classify simple digits like 1 or 0 but performed poorly on complex digits like 5

USING QUANVOLUTION

Architecture of the Quanvolution is as follows :

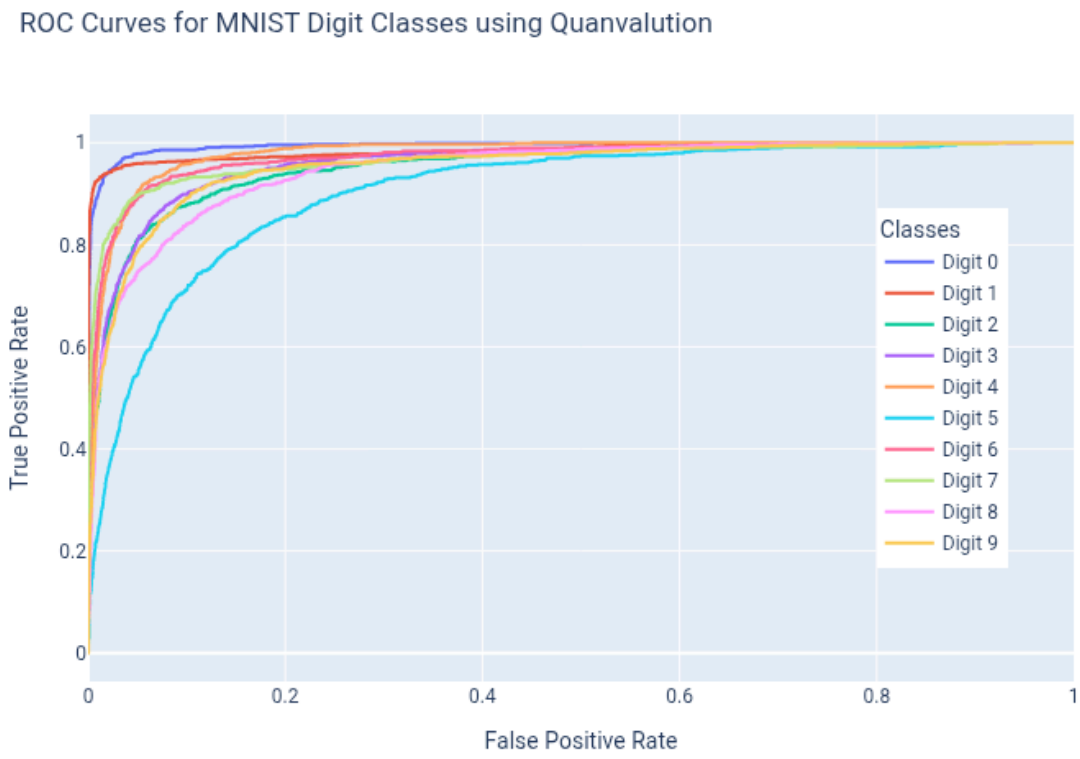
```
        x = self.quanv(x)
        x = self.conv_layers(x)
        x = x.view(x.size(0), -1)
        x = nn.functional.relu(self.fc1(x))
        x = nn.functional.relu(self.fc2(x))
        x = nn.functional.relu(self.fc3(x))
        x = self.out(x)
```

Wherein :

```
self.quanv = QuanvolutionLayer(kernel_size=2, in_channels=1)
```

```
        self.conv_layers = nn.Sequential(
            nn.Conv2d(4, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc1 = nn.Linear(self._to_linear, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 64)
        self.out = nn.Linear(64, 10)
```

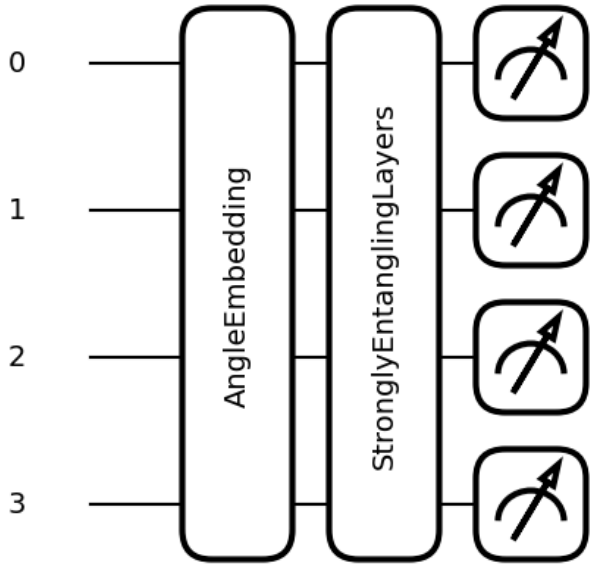
The ROC and Accuracy are as follows



The model performed better than the basic model as this used Quanvolution and the train examples were the same as the basic model . This shows that the Quanvolution models are better for lesser examples

Test Accuracy: 76.02%

The architecture of the Quantum Circuit :



CONCLUSION :

Overall the Quantum models performed poorly as the examples were less and the number of parameters were less and the model eventually over fitted but if the number of qubits were possible increased then it would have performed better